COURS 4 — Les Types Composés en C : les Enregistrements (struct)

Par Robert DIASSÉ

1. Pourquoi a-t-on besoin des structures ?

Jusqu'ici, chaque variable stockait **une seule valeur** d'un seul type :

```
int age = 21;
float note = 14.5;
int annee = 2024;
```

Mais dans la réalité, certaines informations vont naturellement **ensemble**.

Exemples :

- Un étudiant a un **age**, une **note** et une **annee d'inscription**.
- Un produit a un **prix** et une **quantite** en stock.
- Un point géométrique a une **coordonnée x** et une **coordonnée y**.

Avec des variables séparées, il est difficile de regrouper ces informations et de les manipuler ensemble.

La solution en C : les **structures** (`struct`).

---

2. Qu'est-ce qu'une structure ?

Une structure est un **type de données personnalisé** qui regroupe plusieurs variables de types potentiellement différents sous un seul nom.

Chaque variable à l'intérieur d'une structure s'appelle un **champ** (ou membre).

Deux manières existent pour déclarer une structure en C :

- La méthode classique avec le mot-clé `struct`
- La méthode simplifiée avec `typedef`

---

3. Première méthode : `struct`

Format général

```
struct NomDeLaStructure {
    type champ1;
    type champ2;
    type champ3;
};
```

Exemple : déclarer une structure Etudiant

```
struct Etudiant {
    int age;
    float note;
    int annee;
};
```

Explication :

- `struct Etudiant` définit un **nouveau type**
- `age`, `note` et `annee` sont les **champs** de ce type
- Le point-virgule `;` après l'accolade fermante est **obligatoire**

3.1 Déclarer une variable de type struct

Une fois la structure définie, on crée une variable en écrivant le mot `struct` suivi du nom de la structure :

```
struct Etudiant e1;
struct Etudiant e2;
```

Chaque variable (`e1`, `e2`) est un étudiant complet qui contient ses propres champs `age`, `note` et `annee`.

3.2 Accéder aux champs : l'opérateur `.`

Pour lire ou modifier un champ, on utilise l'**opérateur point** `.` :

```
variable.champ
```

Exemple complet :

```c
#include <stdio.h>

struct Etudiant {
    int age;
    float note;
    int annee;
};

int main() {
    struct Etudiant e1;

    e1.age   = 21;
    e1.note  = 14.5;
    e1.annee = 2024;

    printf("Age    : %d\n",   e1.age);
    printf("Note   : %.2f\n", e1.note);
    printf("Annee  : %d\n",   e1.annee);

    return 0;
}
```

Explication :

- `e1.age = 21` → affecte la valeur 21 au champ `age` de la variable `e1`
- `e1.note = 14.5` → affecte 14.5 au champ `note`
- `printf("%d", e1.age)` → affiche le champ `age` de `e1`

Affichage :

```
Age    : 21
Note   : 14.50
Annee  : 2024
```

3.3 Initialisation directe à la déclaration

On peut initialiser tous les champs en une seule ligne, dans l'**ordre de leur déclaration** :

```c
struct Etudiant e1 = {21, 14.5, 2024};
```

- La première valeur va dans `age`, la deuxième dans `note`, la troisième dans `annee`.
- L'ordre doit respecter exactement l'ordre de déclaration des champs.

Exercices – méthode `struct`

1. Déclarer une structure `Point` avec deux champs entiers `x` et `y`. Créer une variable, lui affecter les valeurs `x = 3` et `y = 7`, puis les afficher.
2. Déclarer une structure `Produit` avec un champ `prix` (réel) et un champ `quantite` (entier). Créer deux produits, les initialiser et afficher leurs informations.
3. Déclarer une structure `Compte` avec un champ `numero` (entier) et un champ `solde` (réel). Lire les deux champs au clavier avec `scanf` et les afficher.

---

4. Deuxième méthode : `typedef`

Le problème avec `struct`

Avec la première méthode, on doit **toujours écrire `struct`** avant le nom du type pour déclarer une variable :

```
struct Etudiant e1;   // obligé d'écrire "struct" à chaque fois
struct Etudiant e2;
struct Etudiant e3;
```

C'est répétitif et alourdit le code.

La solution : `typedef`

Le mot-clé `typedef` permet de créer un **alias** pour un type. Appliqué aux structures, il évite de répéter `struct` à chaque déclaration.

Format général avec `typedef`

```
typedef struct {
    type champ1;
    type champ2;
    type champ3;
} NomDuType;
```

La différence avec la première méthode :

- On écrit `typedef struct` au lieu de `struct NomDeStructure`
- Le **nom du type** se place **après l'accolade fermante**, juste avant le `;`

4.1 Exemple : la même structure Etudiant avec typedef

```
typedef struct {
    int age;
    float note;
```

```
    int annee;
} Etudiant;
```

On peut maintenant déclarer des variables **sans écrire** `struct` :

```
Etudiant e1;
Etudiant e2;
```

Au lieu de :

```
struct Etudiant e1;
struct Etudiant e2;
```

4.2 Exemple complet avec typedef

```
#include <stdio.h>

typedef struct {
    int age;
    float note;
    int annee;
} Etudiant;

int main() {
    Etudiant e1;

    e1.age   = 20;
    e1.note  = 16.0;
    e1.annee = 2023;

    printf("Age    : %d\n",   e1.age);
    printf("Note   : %.2f\n", e1.note);
    printf("Annee  : %d\n",   e1.annee);

    return 0;
}
```

Explication :

- `typedef struct { ... } Etudiant` → définit `Etudiant` comme un alias du type structure
- `Etudiant e1` → on déclare `e1` directement, sans écrire `struct` devant
- L'accès aux champs reste identique avec le `.`

L'affichage est identique au programme précédent.

Exercices – méthode `typedef`

1. Reprendre la structure `Point` de l'exercice précédent en utilisant `typedef`. Déclarer une variable, initialiser ses champs et les afficher.
2. Déclarer avec `typedef` une structure `Rectangle` avec deux champs réels `largeur` et `hauteur`. Lire les deux valeurs au clavier, calculer et afficher l'aire et le périmètre.
3. Déclarer avec `typedef` une structure `Compte` avec `numero` (entier) et `solde` (réel). Lire les deux champs, puis afficher un message indiquant si le compte est créditeur (solde > 0) ou débiteur (solde < 0).

---

5. Comparaison des deux méthodes

	`struct` **classique**	`typedef`
Déclaration du type	`struct Etudiant { ... };`	`typedef struct { ... } Etudiant;`
Déclaration d'une variable	`struct Etudiant e1;`	`Etudiant e1;`
Accès aux champs	`e1.age`	`e1.age`
Lisibilité	Plus verbeux	Plus simple et recommandé

Résumé :

- `struct` → méthode de base, valide mais plus lourde à écrire
- `typedef` → méthode recommandée, plus lisible et utilisée en pratique

L'accès aux champs avec `.` est **identique dans les deux cas**.

---

6. Exercices (synthèse)

Exercice A

Déclarer avec `typedef` une structure `Etudiant` contenant `age` (entier) et `note` (réel).

Lire les informations de 3 étudiants au clavier (en utilisant une boucle `for`) et afficher pour chacun son age et sa note.

Exercice B

Déclarer avec `typedef` une structure `Produit` contenant `prix` (réel) et `quantite` (entier).

Créer deux produits en les initialisant directement, puis afficher lequel a le prix le plus élevé en utilisant une structure conditionnelle.

Exercice C

Déclarer avec `typedef` une structure `Compte` avec `numero` (entier) et `solde` (réel).

Lire les informations au clavier. Afficher un menu avec `do...while` :

```
1 - Deposer un montant
2 - Retirer un montant
3 - Afficher le solde
4 - Quitter
```

Pour le retrait, vérifier que le montant ne dépasse pas le solde disponible.

Exercice D

Déclarer avec `typedef` une structure `Point` avec deux champs entiers `x` et `y`.

Lire deux points au clavier. Afficher si les deux points sont identiques ou différents en comparant leurs coordonnées.